

Gabarito da Lista de Exercícios

Formatos de Instrução RISC-V: Montagem e Desmontagem

98800-04 – Fundamentos de Sistemas Computacionais · Prof. Iaçã Ianiski Weber

	Palavra	opcode	Formato	Instrução
1. a)	0x00C58533	0110011	R	add (add a0, a1, a2)
b)	0xFFF28293	0010011	I	addi (addi t0, t0, -1)
c)	0x00A12423	0100011	S	sw (sw a0, 8(sp))
d)	0xFE031CE3	1100011	B	bne (bne t1, zero, -8)
e)	0x000115B7	0110111	U	lui (lui a1, 0x11)
f)	0x00C000EF	1101111	J	jal (jal ra, +12)
g)	0x00028067	1100111	I	jalr (jalr zero, t0, 0)
h)	0x0006A803	0000011	I (load)	lw (lw a6, 0(a3))

	Instrução	Campos (binário)	Hex
2. a)	add x5, x6, x7	0000000 00111 00110 000 00101 0110011	0x007302B3
b)	addi t2, t2, -4	11111111100 00111 000 00111 0010011	0xFFC38393
c)	lw a0, 8(sp)	000000001000 00010 010 01010 0000011	0x00812503
d)	sw s1, 12(s0)	0000000 01001 01000 010 01100 0100011	0x00942623
e)	xor a1, a1, a1	0000000 01011 01011 100 01011 0110011	0x00B5C5B3
f)	srai t0, t1, 2	0100000 00010 00110 101 00101 0010011	0x40235293

Item (e): $x \text{ XOR } x = 0$ para qualquer valor, logo a instrução **zera a1**.

3. a) $0x40B50533 = \text{sub } a0, a0, a1$ (funct7 = 0100000)
 b) $0x00F37313 = \text{andi } t1, t1, 15$ (isola o *nibble* menos significativo)
 c) $0xFFC62A83 = \text{lw } s5, -4(a2)$ (imediato negativo)
 d) $0xFE112E23 = \text{sw } ra, -4(sp)$. É um prólogo de função: salva o endereço de retorno (ra) na pilha antes de chamar outra função.
 e) $0x00050593 = \text{addi } a1, a0, 0 = \text{pseudoinstrução mv } a1, a0$.
4. a) $0x05200513 = \text{addi } a0, \text{zero}, 82 \rightarrow 'R'$
 b) $0x05600593 = \text{addi } a1, \text{zero}, 86 \rightarrow 'V'$
 c) $0x03300613 = \text{addi } a2, \text{zero}, 51 \rightarrow '3'$
 d) $0x03200693 = \text{addi } a3, \text{zero}, 50 \rightarrow '2'$

Palavra formada: RV32.

5. a) Deslocamento = $0x110 - 0x104 = +12$ bytes. Em 13 bits: 00 000000 0110 0.
 $\text{imm}[12]=0, \text{imm}[10:5]=000000, \text{imm}[4:1]=0110, \text{imm}[11]=0$.
 Campos: 0000000|00000|00101|000|01100|1100011 $\Rightarrow 0x00028663$.
- b) Deslocamento = $0x100 - 0x10C = -12$ bytes = 1 11111111 1 1111111010 0 (21 bits, compl. de 2).
 $\text{imm}[20]=1, \text{imm}[10:1]=1111111010, \text{imm}[11]=1, \text{imm}[19:12]=11111111$.
 Campos: 1|1111111010|1|11111111|00000|1101111 $\Rightarrow 0xFF5FF06F$.
- c) beq (tipo B): $\pm 2^{10} = \pm 1024$ instruções (± 4 KiB). jal (tipo J): $\pm 2^{18} \approx \pm 262\,000$ instruções (± 1 MiB).
 d) Não alcança ($3000 > 1024$). Inverte-se a condição e usa-se um salto incondicional, que alcança $\pm 2^{18}$ instruções:
 bne t0, zero, Continua ; jal zero, Achei ; Continua: ...

6. a) Desmontagem:

0x00	0x00053023	inválida (ver abaixo)
0x04	0x00450513	addi a0, a0, 4
0x08	0xFFF58593	addi a1, a1, -1
0x0C	0xFE059AE3	bne a1, zero, -12 (volta para 0x00)
0x10	0x00008067	jalr zero, ra, 0 (= ret)

A palavra em 0x00 tem opcode 0100011 (store), mas funct3 = 011 **não existe** para o opcode de store (sb=000, sh=001, sw=010).

- b) O **bit 12** (bit menos significativo do funct3) foi invertido: 010 $\rightarrow$ 011. A palavra original era $0x00052023 = \text{sw } zero, 0(a0)$.
 c) Rotina corrigida:

```

Loop:  sw    zero, 0(a0)
        addi a0, a0, 4
        addi a1, a1, -1
        bne  a1, zero, Loop
        jalr zero, ra, 0    # ret

```

7. a) $0x00008067 = \text{jalr zero, ra, 0} = \text{pseudoinstrução ret}$.
 b) Toda função retorna ao chamador saltando para o endereço salvo em **ra** ($PC = ra + 0$); como **rd** = **zero**, o endereço de retorno do próprio salto é descartado.
 c) +32 bytes: $\text{imm}[20]=0$, $\text{imm}[10:1]=0000010000$, $\text{imm}[11]=0$, $\text{imm}[19:12]=00000000$, $\text{rd}=00001 \Rightarrow 0x020000EF$.
 d) B e J somam o deslocamento ao **PC**, que aponta sempre para endereços pares; o bit 0 seria sempre 0, então não é armazenado e ganha-se 1 bit de alcance. O **jalr** soma o imediato a **rs1**, um valor **arbitrário** em bytes, e reaproveita o formato I exatamente como os loads ($\text{rs1} + \text{imm}$); por isso o imediato é em bytes, sem multiplicação.
8. a) Os 12 bits baixos ($0x678$) têm bit 11 = 0, sem pegadinha:
 $\text{lui t0, } 0x12345 \text{ (} 0x123452B7 \text{); addi t0, t0, } 0x678 \text{ (} 0x67828293 \text{)}.$
 b) Abordagem direta: $\text{lui t1, } 0xDEADB + \text{addi t1, t1, } 0xEEF$. Mas $0xEEF$ tem bit 11 = 1: o **addi** estende o sinal e soma -273 , resultando $0xDEADB000 - 273 = 0xDEADAEEF \neq 0xDEADBEEF$.
 Correta: $\text{lui t1, } 0xDEADC \text{ (} 0xDEADC337 \text{); addi t1, t1, } -273 \text{ (} 0xEEF30313 \text{);}$
 $0xDEADC000 - 273 = 0xDEADBEEF$.
 c) Se o bit 11 dos 12 bits baixos for 1, o **addi** subtrai $0x1000$ do resultado; compensa-se somando +1 ao imediato de 20 bits do **lui**. (A pseudoinstrução **li** faz isso automaticamente.)

#	Veredito	O que o hex realmente faz	Correção
1	Errado	$0x40990433 = \text{sub s0, s2, s1}$	$0x41248433$
2	Errado	$0x28512023 = \text{sw t0, 640(sp)}$	$0x00512A23$
3	Errado	$0x00B50163 = \text{beq a0, a1, +2}$	$0x00B50463$
4	Correto	$0x00655383 = \text{lhu t2, 6(a0)}$	—

9. 1. **rs1** e **rs2** foram trocados: o código calcula $s2 - s1$ em vez de $s1 - s2$. Subtração não é comutativa.
 2. O imediato do formato S não foi **dividido**: $20 = 10100$ foi colocado inteiro em $\text{imm}[11:5]$, e o offset virou $20 \times 32 = 640$. Correto: $\text{imm}[11:5]=00000000$, $\text{imm}[4:0]=10100$.
 3. O deslocamento foi contado em **instruções** (2) em vez de **bytes** (8); +2 aponta para o meio da instrução seguinte.
 4. Nada a corrigir.
10. Itens (a) e (b), desmontagem com rótulos:
- ```

0x200 addi t0, zero, 0 # acumulador = 0
0x204 Loop: beq a1, zero, Fim # +16 → 0x214
0x208 add t0, t0, a0 # acumula a0
0x20C addi a1, a1, -1 # decrementa contador
0x210 jal zero, Loop # -12 → 0x204
0x214 Fim: addi a0, t0, 0 # a0 = resultado (mv)

```
- c) Soma **a0** repetidamente, **a1** vezes: calcula a **multiplicação**  $a0 \times a1$  por somas sucessivas.  
 d)  $6 \times 7 = 42$ .  
 e) O **beq** é avaliado a cada iteração ( $a1 = 7, 6, \dots, 1$ , falso) e uma última vez com  $a1 = 0$  (verdadeiro):  $7 + 1 = 8$  vezes.
11. a) Com **rs1/rs2/rd** sempre nos mesmos bits, o processador pode ler o banco de registradores *em paralelo* com a decodificação, antes mesmo de saber qual instrução é. Isso exige menos multiplexadores e simplifica o hardware.  
 b) O **store** precisa de dois registradores-fonte e um imediato, mas não tem **rd**. Dividir o imediato permite manter **rs1** e **rs2** exatamente nas posições dos formatos R/I: os 5 bits baixos do imediato ocupam o lugar onde estaria o **rd**. Sacrifica-se a contiguidade do imediato (menos crítico) para preservar a posição dos registradores (mais crítico).  
 c) Instruções nunca começam em endereço ímpar, então o bit 0 do deslocamento seria sempre 0 e armazená-lo desperdiçaria 1 bit: múltiplos de 2 **dobram o alcance** e mantêm o suporte à extensão comprimida de 16 bits. Múltiplos de 4 dobrariam o alcance de novo, mas quebrariam esse suporte.